

Embedded Systems Hardware

1. CORE PROCESSING UNIT



- Microcontrollers (MCU)
- Microprocessors (MPU)
- Digital Signal Processors (DSP)
- System on Chip (SoC)

Examples:



4. OUTPUT DEVICES



2. MEMORY



3. INPUT DEVICES



5. COMMUNICATION INTERFACES



6. POWER MANAGEMENT



7. CLOCK & TIMING



8. DEVELOPMENT & DEBUGGING



9. OTHER HARDWARE COMPONENTS



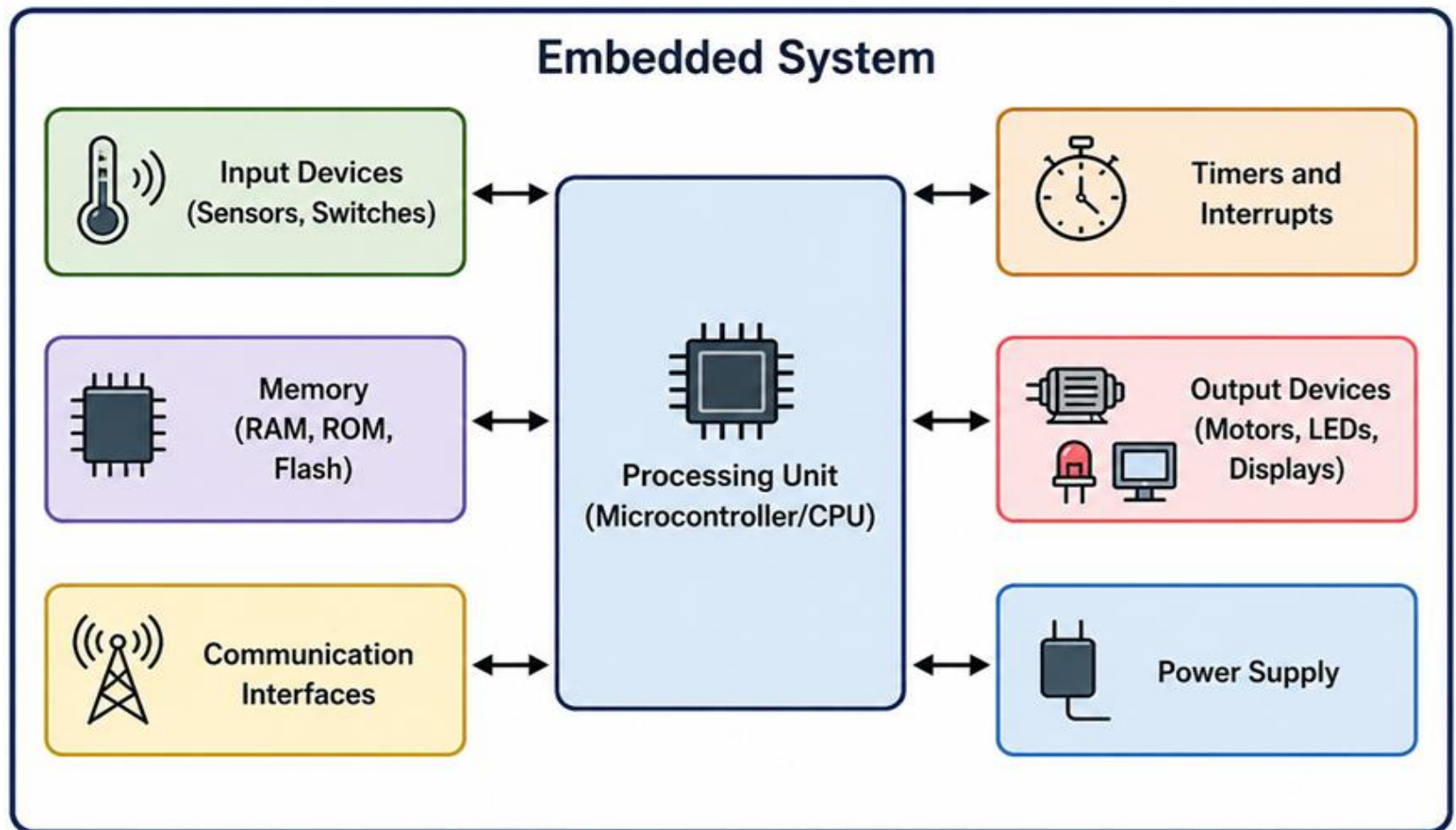
Prof Ben B. K. Ayawli

Introduction to Embedded Hardware

- Embedded hardware consists of electronic components that perform the following functions:
 - Sensing
 - Processing
 - Communication
 - Storage
 - Control

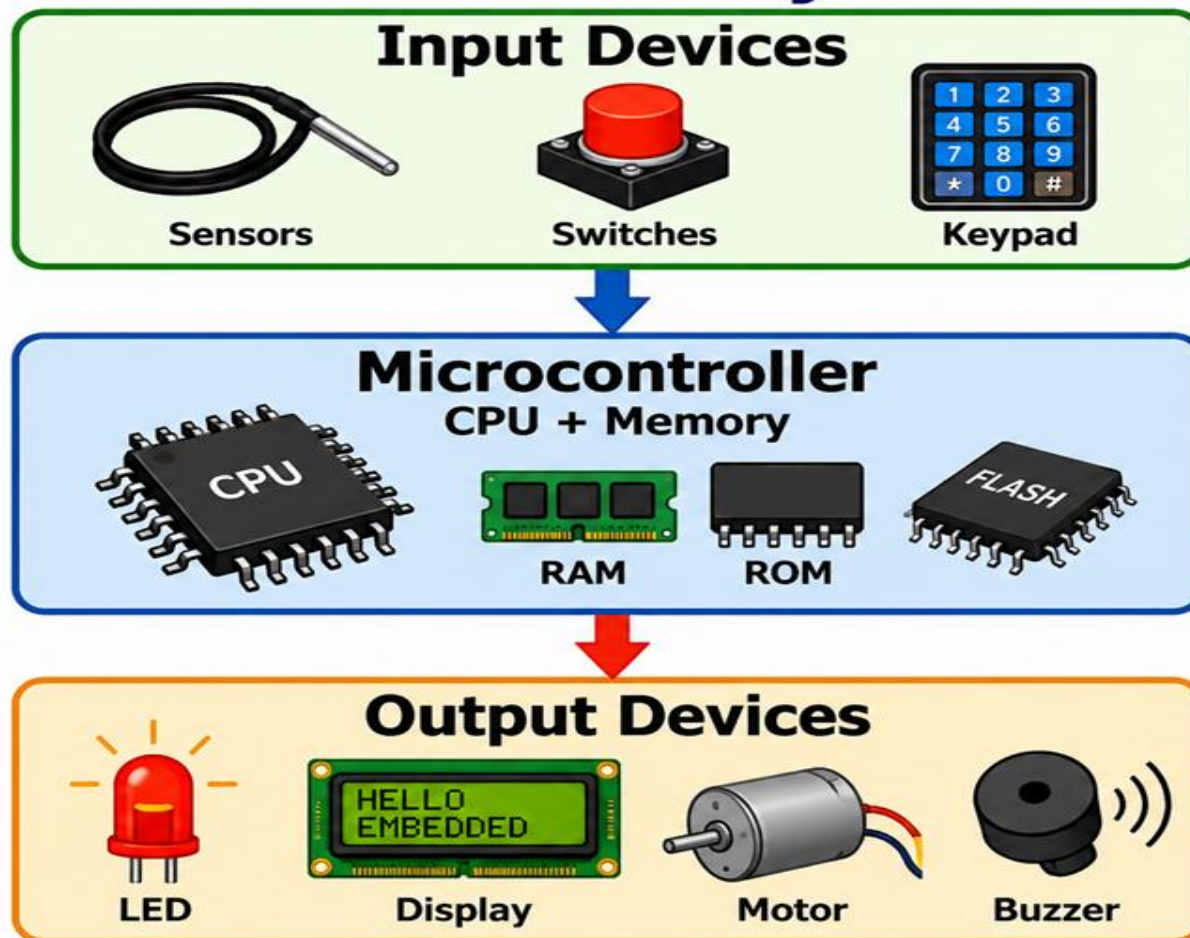
Introduction to Embedded Hardware (cont)

- A typical embedded system



Embedded System Hardware Architecture

- A simplified architecture



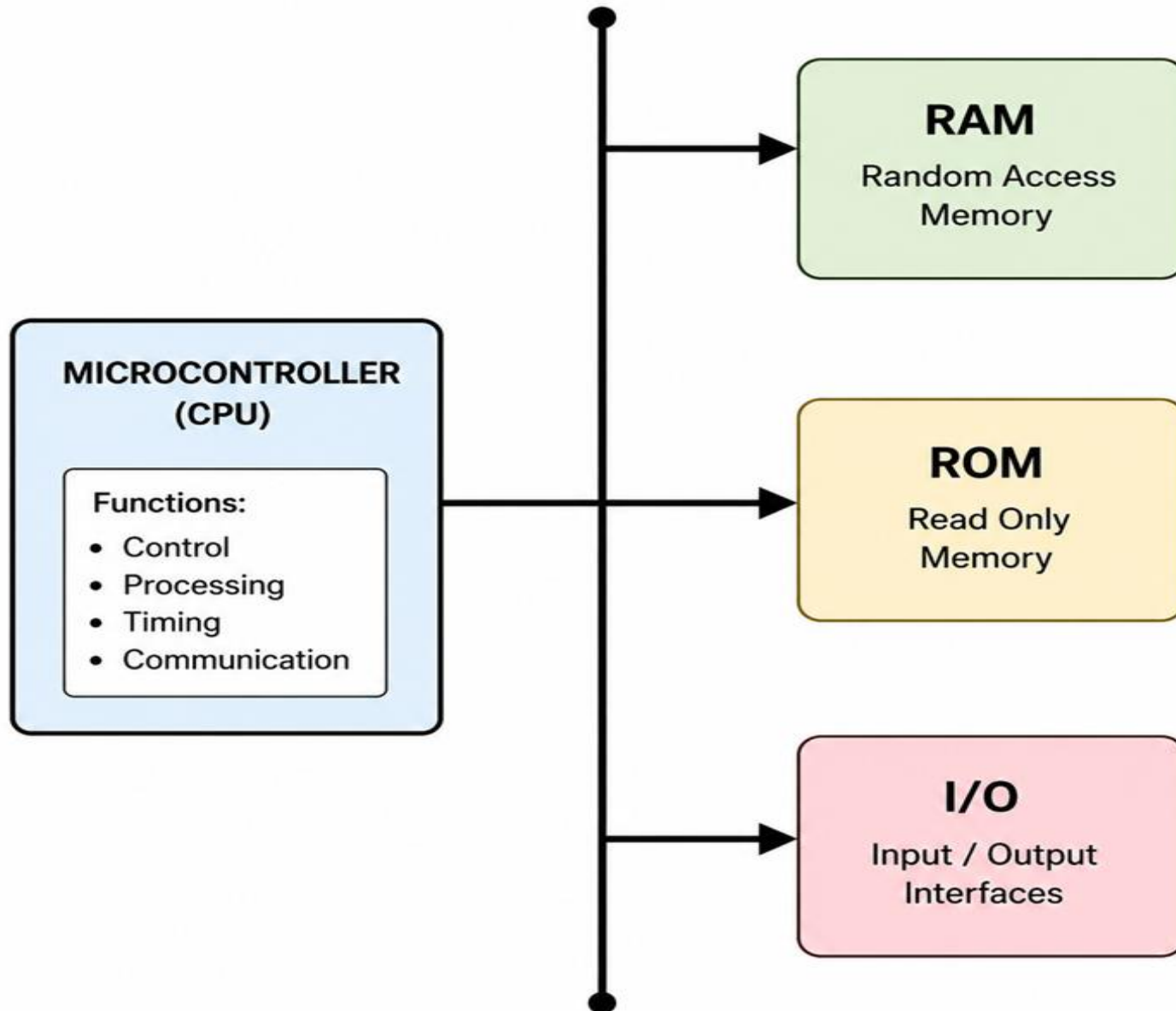
Microprocessors versus Microcontrollers

Microprocessors versus Microcontrollers

- **Microprocessor**

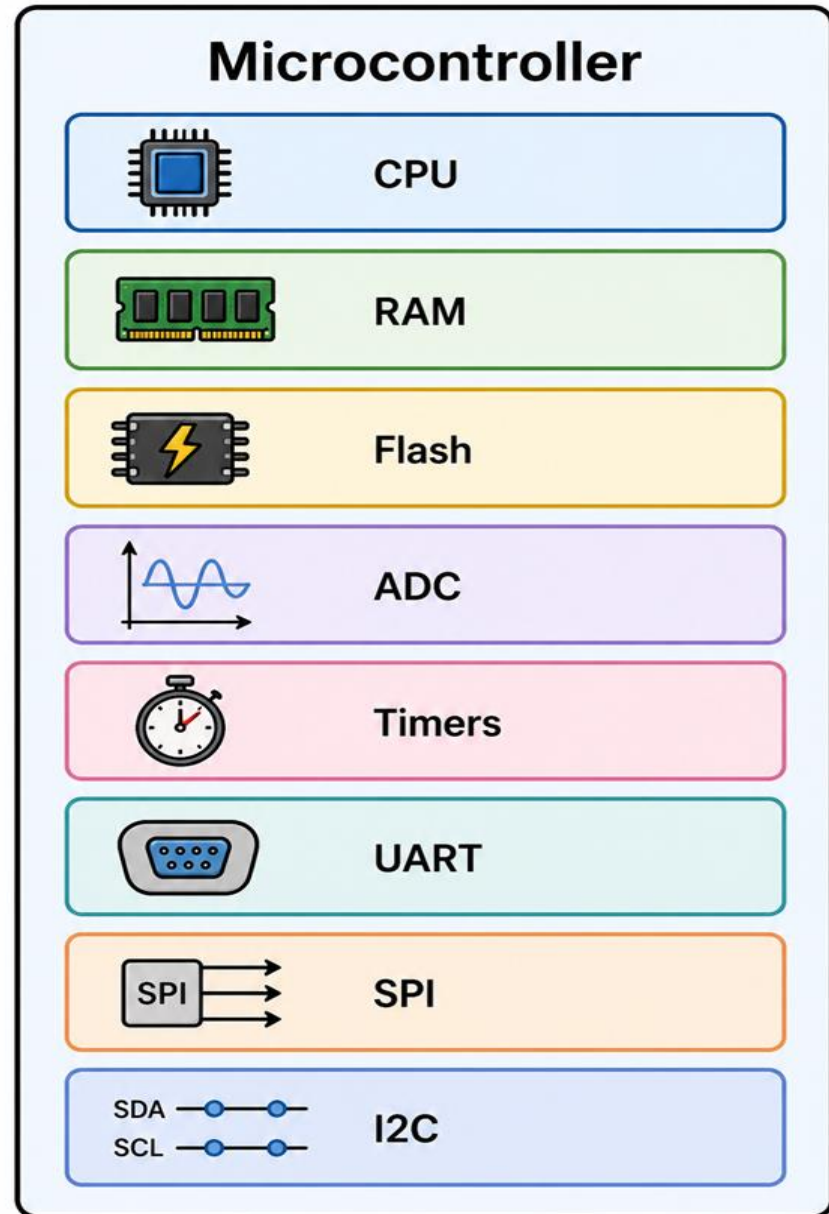
- A microprocessor contains only the CPU.
- Examples:
 - Intel Core i7
 - AMD Ryzen
 - Intel Xeon
- Requires external:
 - RAM
 - ROM
 - I/O ports

Architecture of CPU



Microcontroller

- A microcontroller integrates everything on one chip.
- Components:
 - CPU
 - RAM
 - ROM
 - Timers
 - ADC
 - Communication interfaces



Microcontroller

- Examples of microcontroller
 - ATmega328P
 - ESP32
 - STM32
 - PIC16F877A

Microcontroller (cont)

ATmega328P



- 8-bit AVR RISC microcontroller
- 32 KB Flash
- 2 KB SRAM
- 1 KB EEPROM
- 23 I/O Pins
- UART, SPI, I2C
- 10-bit ADC
- Operates at 1.8V – 5.5V
- Max Speed: 20 MHz

ESP32



- 32-bit Xtensa® dual-core microcontroller
- Wi-Fi 802.11 b/g/n
- Bluetooth 4.2 / BLE
- 520 KB SRAM
- 4 MB Flash (typ.)
- 34 GPIO Pins
- UART, SPI, I2C, I2S, CAN, ADC, DAC
- Operates at 2.2V – 3.6V
- Max Speed: 240 MHz

STM32



- 32-bit ARM® Cortex®-M3 microcontroller
- Up to 72 MHz clock
- 64 KB – 512 KB Flash
- 20 KB – 64 KB SRAM
- 37 I/O Pins
- UART, SPI, I2C, CAN, USB, ADC, DAC
- Operates at 2.0V – 3.6V
- Wide range of packages and features

PIC16F877A



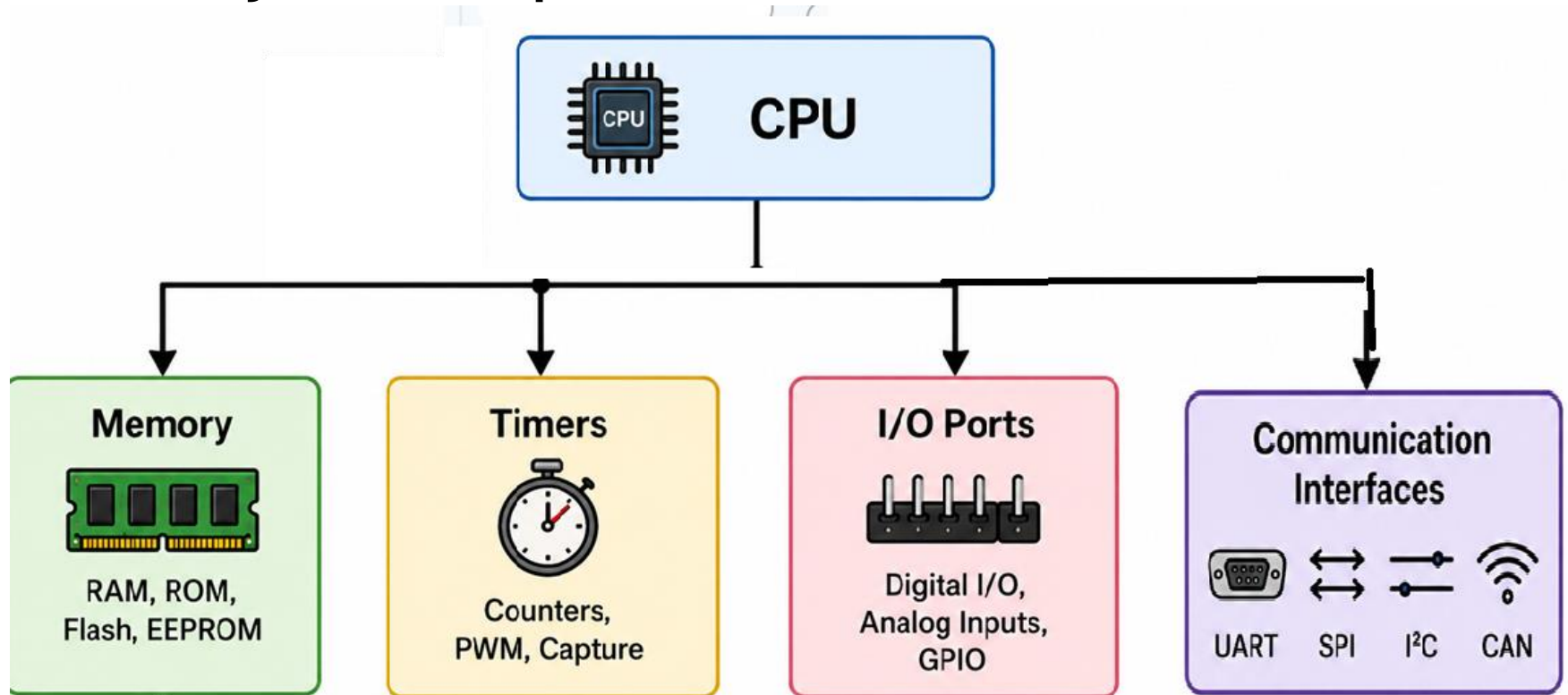
- 8-bit RISC microcontroller
- 8 KB Flash
- 368 Bytes RAM
- 256 Bytes EEPROM
- 33 I/O Pins
- UART, SPI, I2C, CCP, ADC, PWM
- Operates at 2.0V – 5.5V
- Max Speed: 20 MHz

Microprocessors versus Microcontrollers

Feature	Microprocessor	Microcontroller
RAM	External	Internal
ROM	External	Internal
Cost	High	Low
Power Consumption	High	Low
Applications	Computers	Embedded Systems

Microcontroller Architecture

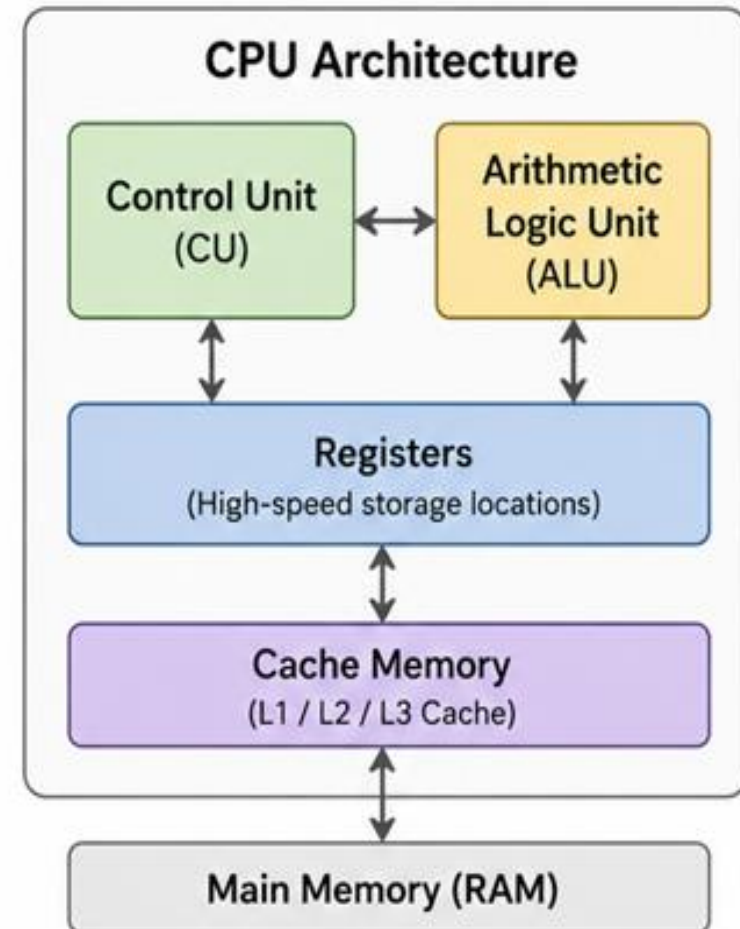
- The microcontroller is the brain of an embedded system.
- Major components:



Central Processing Unit (CPU)

- The CPU performs:
 - Arithmetic operations
 - Logical operations
 - Program execution
 - Control operations
- Components:
 - Arithmetic Logic Unit (ALU)
 - Control Unit
 - Registers

Central Processing Unit (CPU) (cont)



Arithmetic Logic Unit (ALU)

- Performs:
 - Addition
 - Subtraction
 - Multiplication
 - Division
 - Logical operations

Control Unit

- Responsible for:
 - Fetching instructions
 - Decoding instructions
 - Executing instructions

Registers

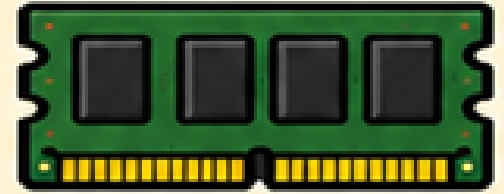
- Small memory locations used for temporary storage.
- Examples:
 - Accumulator
 - Program Counter
 - Stack Pointer
 - Status Register

Embedded Memory Systems

- Memory stores instructions and data.
- Types include:
 - RAM
 - ROM
 - Flash
 - EEPROM

Random Access Memory (RAM)

- Stores temporary data
- Stores variables
- Characteristics:
 - Fast
 - Volatile
 - Read/Write
- Example:
 - Variables in C are stored in RAM.
 - `int temperature = 30;`

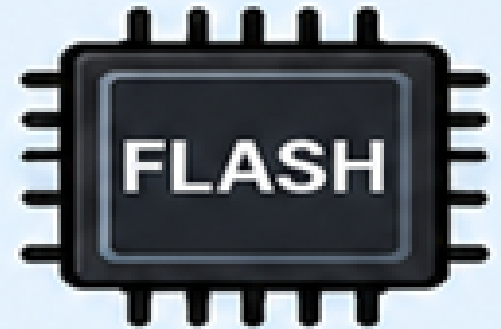


Read Only Memory (ROM)

- Stores permanent programs.
- Characteristics:
 - Non-volatile
 - Read-only

Flash Memory

- Stores firmware/program
- Advantages:
 - Non-volatile
 - Electrically erasable
- Examples:
 - Arduino program storage
 - Smartphone firmware



EEPROM

- Electrically Erasable Programmable Read-Only Memory
- Stores permanent data
- Used to store:
 - Configuration settings
 - Calibration data
- Example:
 - Robot speed settings



System Bus Architecture

- Communication inside a microcontroller occurs through buses
- Types:
 - Data Bus
 - Address Bus
 - Control Bus

System Bus Architecture (cont)

- **Data Bus**

- Transfers data
- Example:
 - Sensor data to CPU

- **Address Bus**

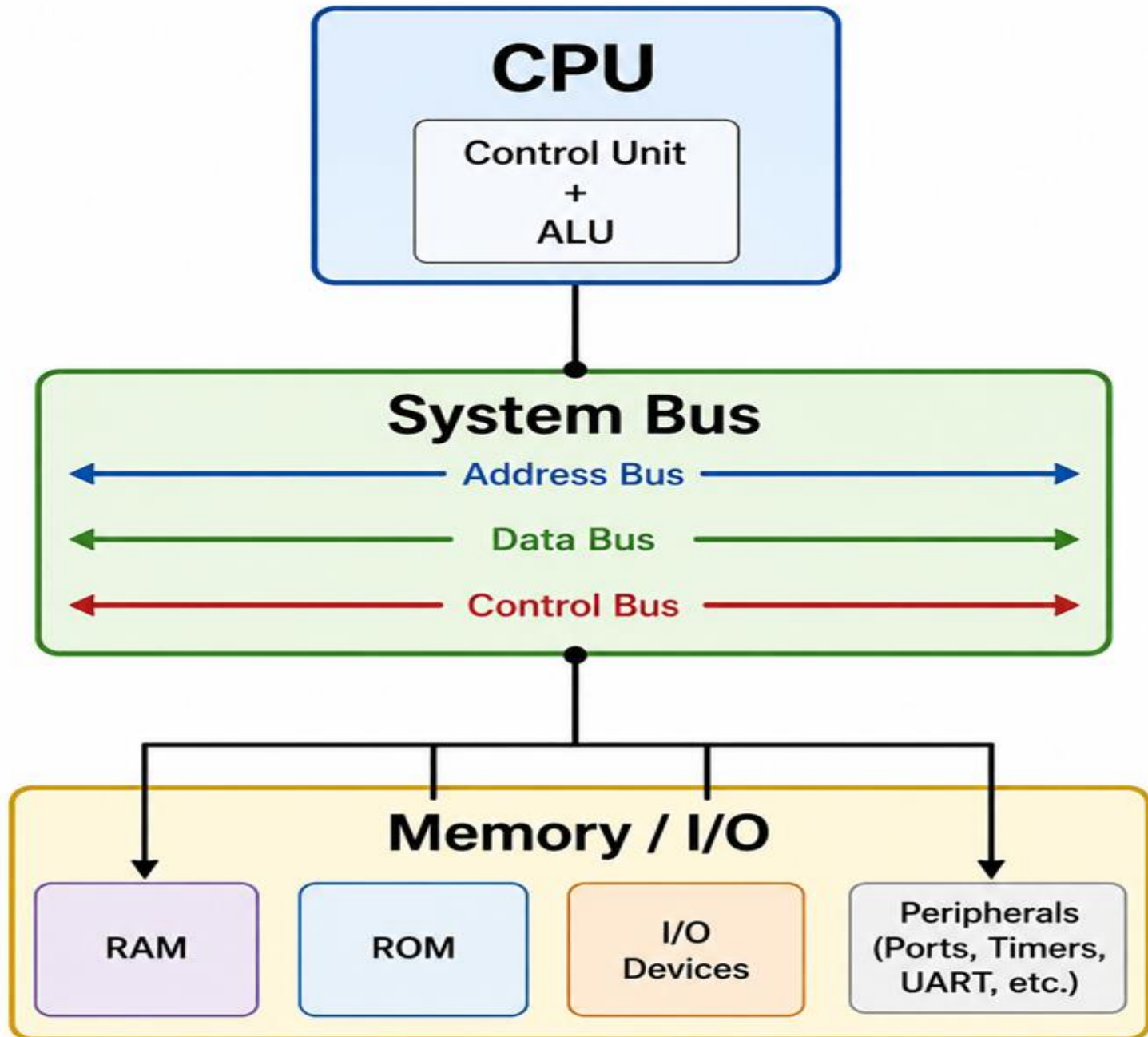
- Specifies memory locations.
- Example:
 - Address 0x1000

System Bus Architecture (cont)

- **Control Bus**

- Carries control signals
- Examples:
 - Read
 - Write
 - Interrupt

Bus Architecture Diagram



General Purpose Input Output (GPIO)

- **GPIO** pins allow communication with external devices.
- Examples:
 - Reading switches
 - Driving LEDs
 - Reading sensors

GPIO Modes

- **Input Mode**

- Reads external signals

- **Example:**

- Button

```
pinMode(2, INPUT);
```

- **Output Mode**

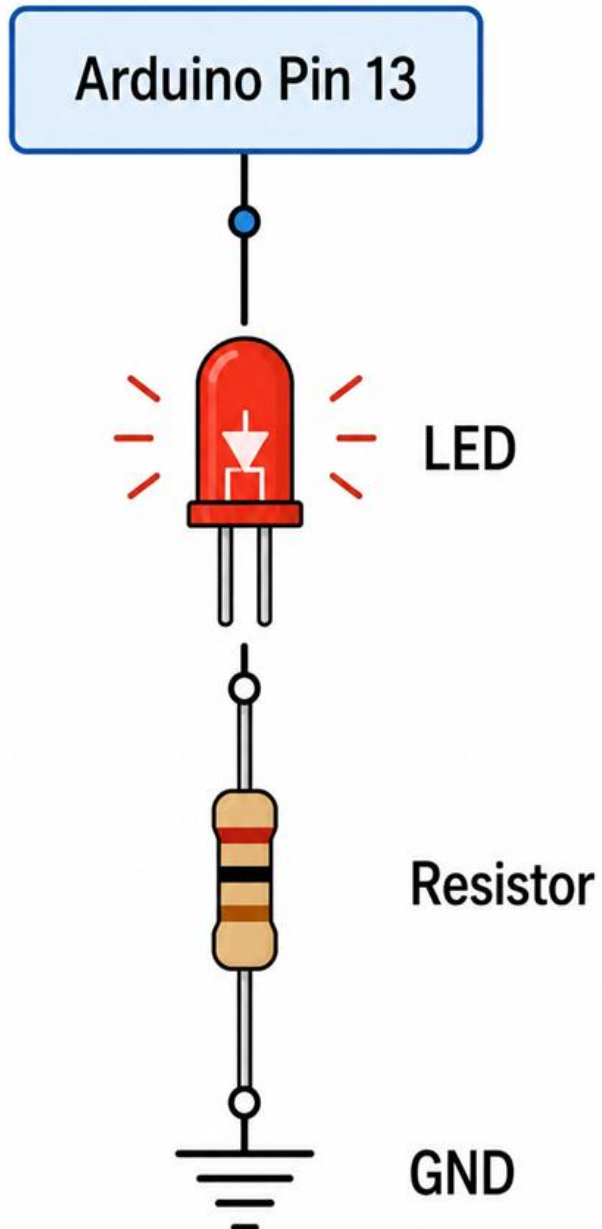
- Drives external devices

- **Example:**

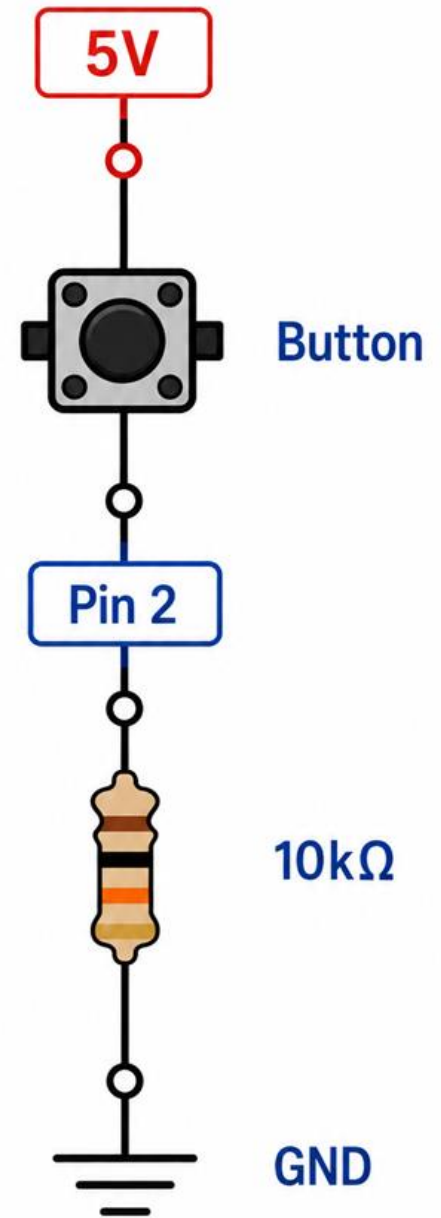
- LED

```
pinMode(13, OUTPUT);
```

LED Interface Example



Push Button Interface

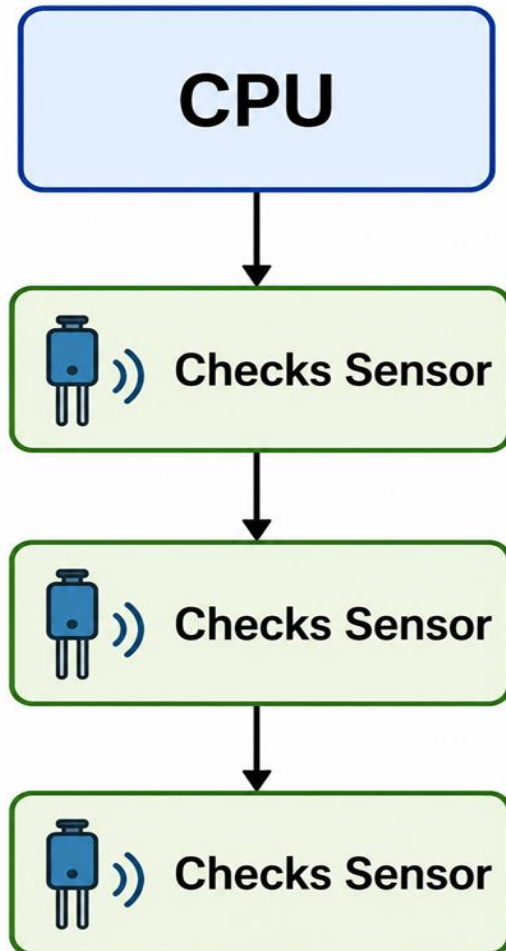


Interrupt System

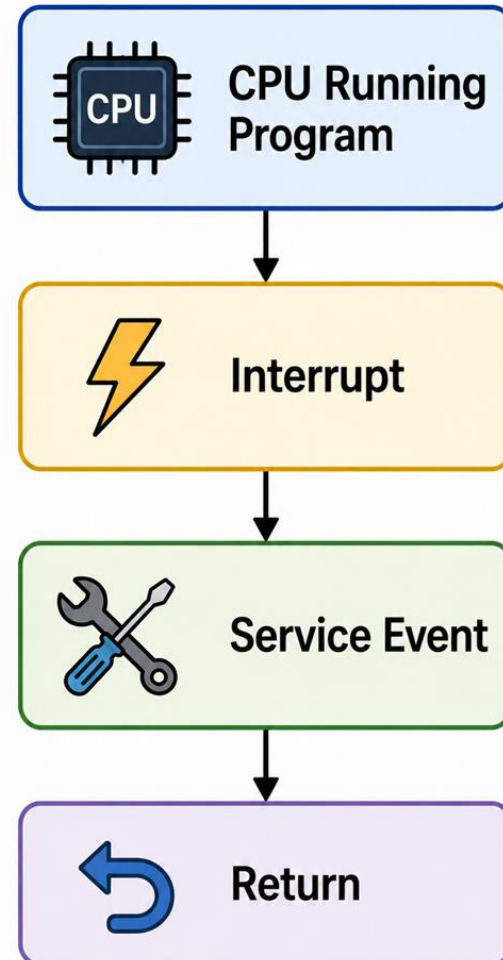
- Interrupts allow the CPU to respond immediately to events.
- **Without interrupts:** Wastes processing time.

Interrupt System (cont)

- Without Interrupt



- With Interrupt



Interrupt Sequence

1. Event occurs
2. Interrupt request generated
3. CPU pauses current task
4. ISR executes
5. CPU resumes

Interrupt Service Routine (ISR)

- Example:

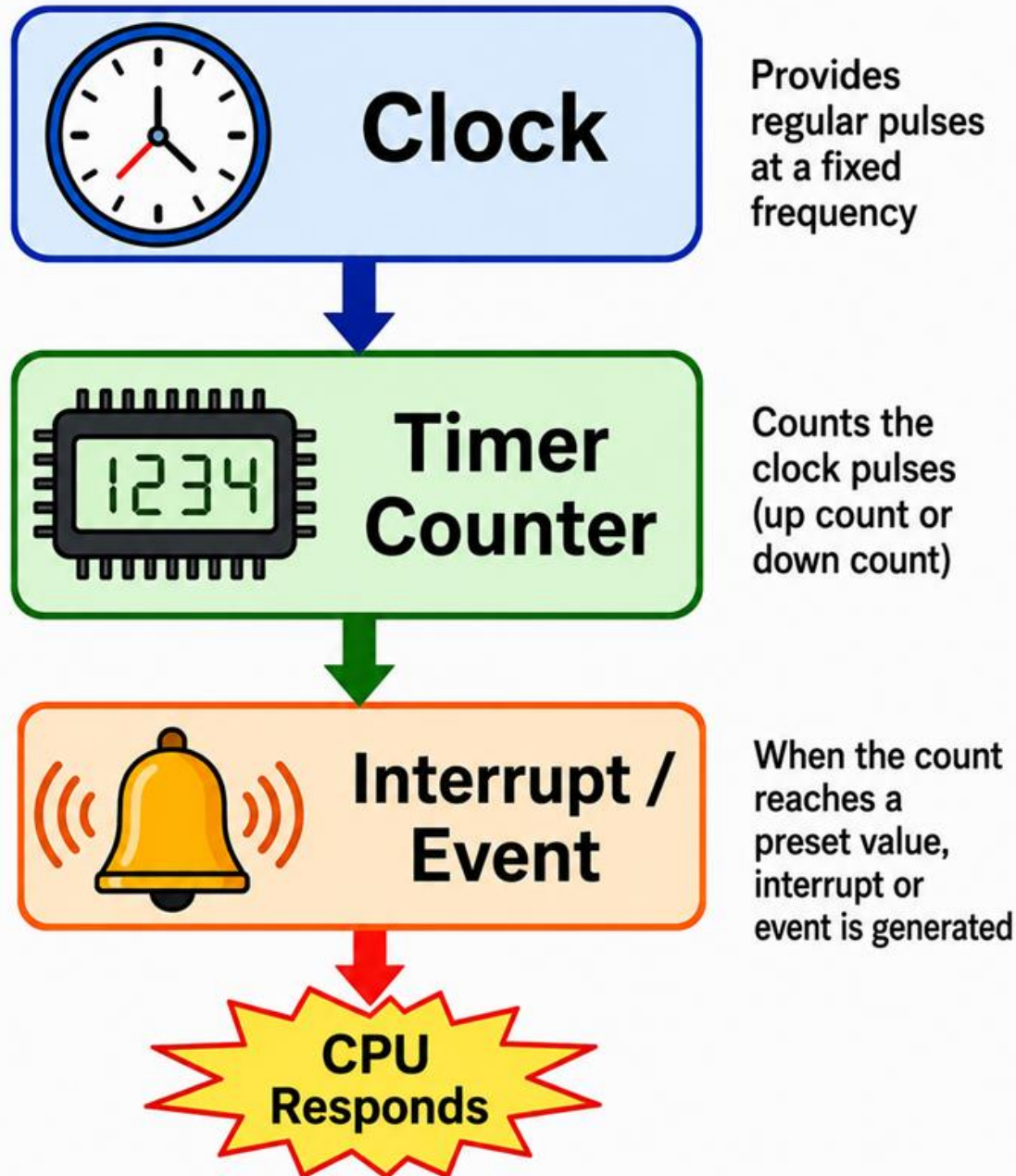
```
void ISR_Button()  
{  
    counter++;  
}
```

- Applications:
 - Emergency stop
 - Encoder counting
 - Communication

Timers and Counters

- Timers generate accurate delays.
- Applications:
 - PWM generation
 - Robot control
 - Motor speed measurement

Timer Operation



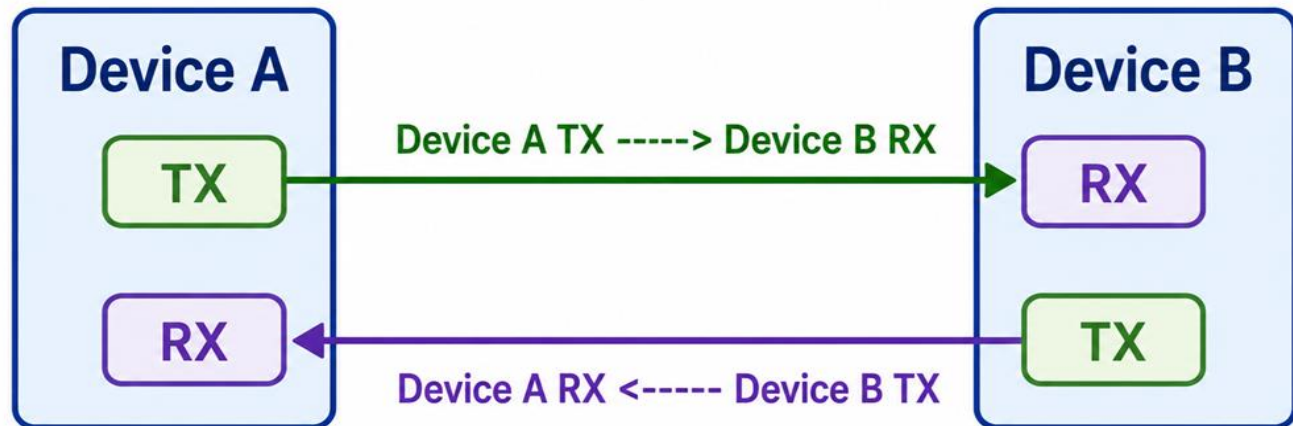
Communication Interfaces

- Modern embedded systems communicate with **sensors** and **peripherals**
- Common interfaces:

Interface	Speed	Distance
UART	Medium	Long
SPI	High	Short
I²C	Medium	Short
CAN	High	Medium
USB	Very High	Medium

UART Communication

- Universal Asynchronous Receiver Transmitter
- Uses:
 - Arduino serial monitor
 - GPS modules
 - Bluetooth modules
- Connection



UART Communication (cont)

- Arduino Example

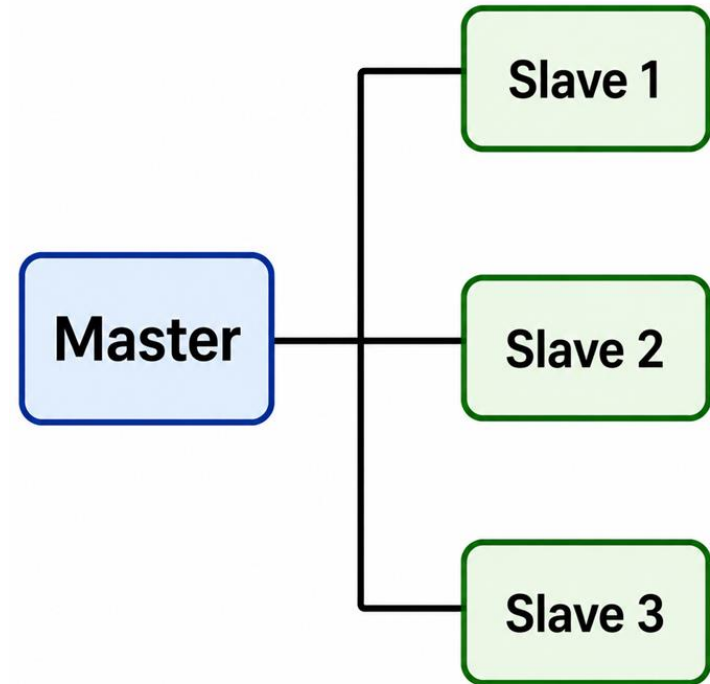
```
Serial.begin(9600);
```

```
Serial.println("Hello World");
```


SPI Communication

- Serial Peripheral Interface
- Lines:
 - MOSI
 - MISO
 - SCK
 - SS

Architecture



SPI Communication (cont)

-  **MOSI (Master Out, Slave In)**
 - Data line used to transmit data from the master device to the slave device.
-  **MISO (Master In, Slave Out)**
 - Data line used to transmit data from the slave device to the master device.

SPI Communication (cont)

-  **SCK (Serial Clock)**
 - Clock signal generated by the master device to synchronize data transmission.
-  **SS (Slave Select)**
 - Control signal used by the master device to select and communicate with a specific slave device. (Also called **CS** – **Chip Select** in some systems.)

SPI Communication (cont)

- Applications:
 - SD cards
 - LCDs
 - Sensors

I²C Communication

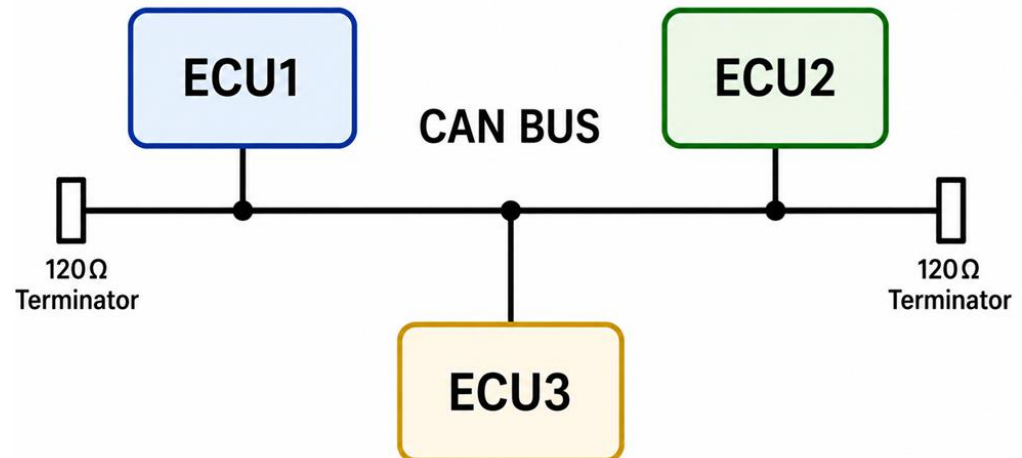
- Inter-Integrated Circuit
- Requires only two wires:
 -  **SDA (Serial Data Line)**
 - The bidirectional data line used to transmit and receive data between devices in the I²C communication protocol.
 -  **SCL (Serial Clock Line)**
 - The clock line is used to synchronize data transfer between devices in the I²C communication protocol
 - The master device usually generates the clock signal.

I²C Communication (cont)

- Advantages
 - Few wires
 - Multiple devices

CAN Bus

- Controller Area Network
- Widely used in vehicles
- Applications:
 - Engine control
 - ABS
 - Airbag systems
- Architecture:

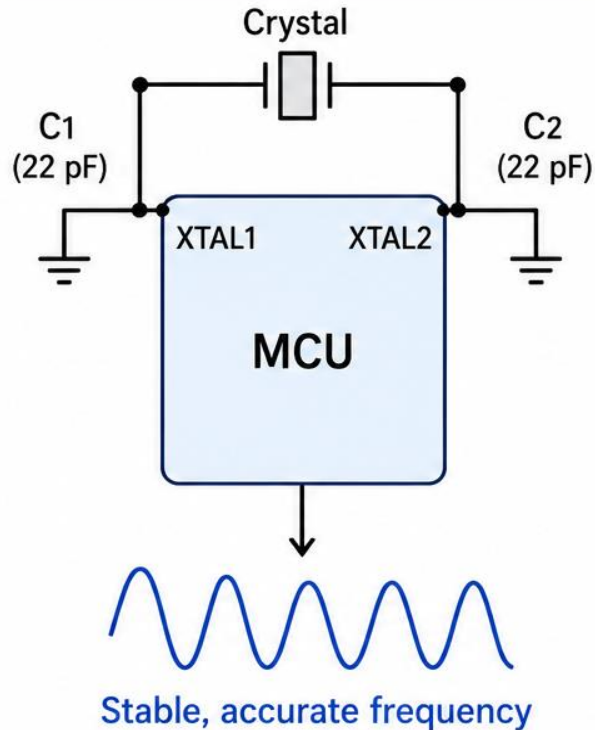


Clock Systems

- The clock synchronizes all operations.
- Clock Sources:
 - Crystal oscillator
 - RC oscillator
 - External clock

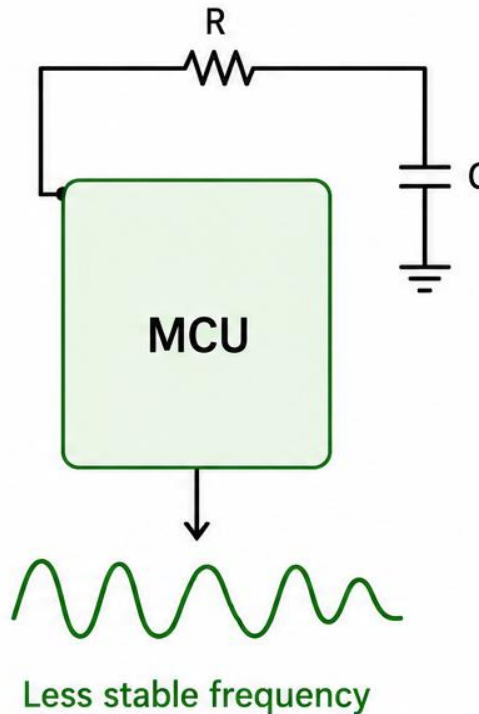
Clock Systems (cont)

1. Crystal Oscillator



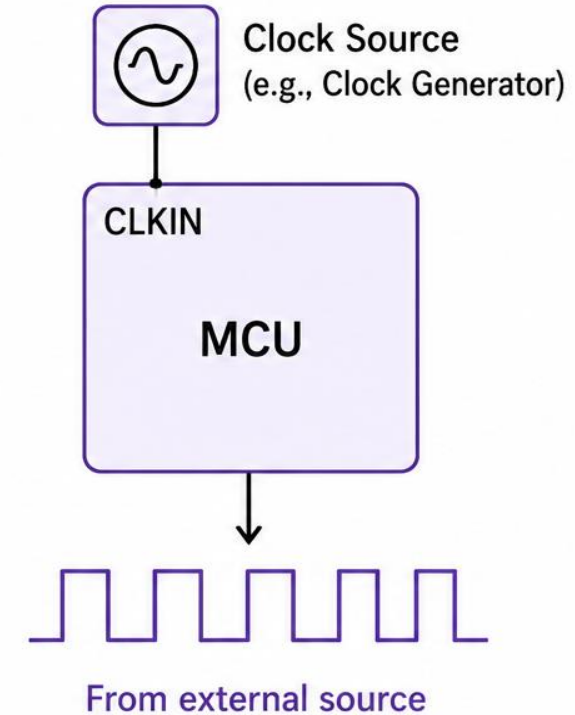
- Very accurate and stable
- Low drift with temperature
- Used in precision applications

2. RC Oscillator



- Simple and low cost
- Less accurate
- Affected by temperature and component tolerance

3. External Clock



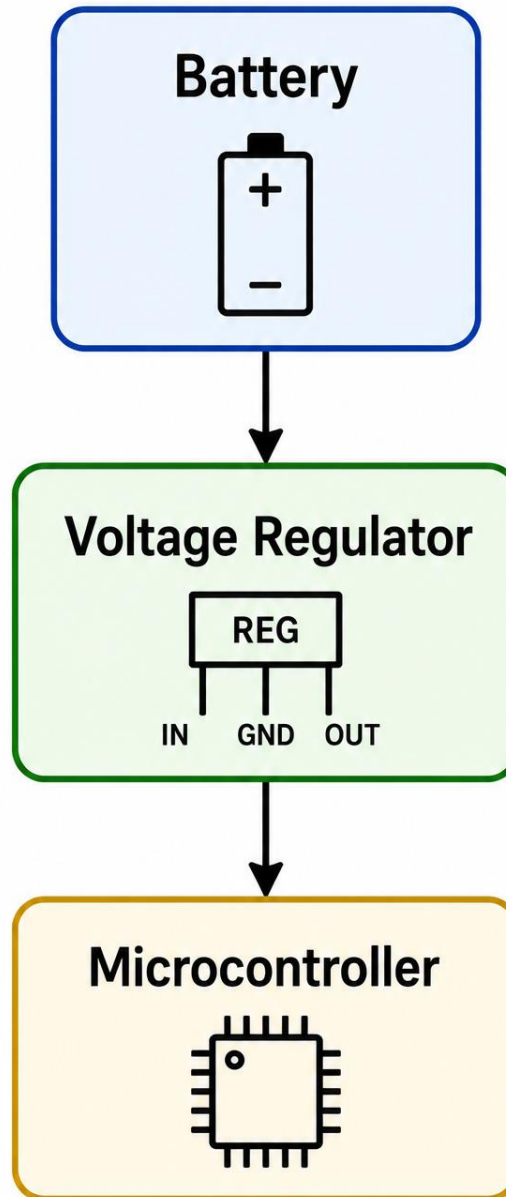
- Frequency defined by external source
- Very accurate (depends on external source)
- Used when synchronization is required

Power Supply Systems

- Embedded systems require stable power.
- Common voltages

Device	Voltage
Arduino Uno	5V
ESP32	3.3V
Raspberry Pi	5V

Power Supply Block



Hardware Design Considerations

- When selecting hardware Consider:
 - Cost
 - Processing speed
 - Power consumption
 - Memory
 - Reliability
 - Environmental conditions

Case Study

- Arduino-Based Smart Irrigation System
- Components:

Component	Purpose
Arduino Uno	Controller
Soil Sensor	Moisture Detection
Relay	Pump Control
Water Pump	Irrigation
LCD	Display

Case Study (cont)

- **Operation:**

- Read soil moisture.
- Compare with threshold.
- Activate pump if dry.
- Stop pump when moisture improves.

Laboratory Exercise: Interfacing an LED

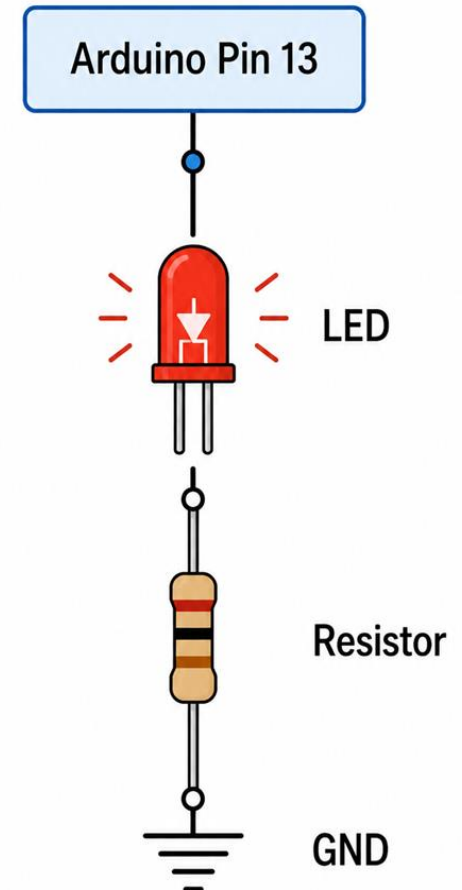
- Program

```
void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
  delay(1000);

  digitalWrite(13, LOW);
  delay(1000);
}
```

LED Interface Example



Laboratory Exercise: Reading a Push Button

```
int button = 2;

void setup()
{
  pinMode(button, INPUT);
  Serial.begin(9600);
}

void loop()
{
  int state = digitalRead(button);

  Serial.println(state);

  delay(100);
}
```


Assignment

Question 1

- a. Differentiate a microprocessor from a microcontroller
- b. Explain the functions of RAM, ROM, Flash, and EEPROM
- c. What is GPIO?
- d. Explain the purpose of interrupts.

Question 2

Design a smart streetlight hardware system.

Question 3

Design an embedded system for automatic water-level monitoring